VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING




## Computer Networking

---

### Assignment Report

# Assignment 1 : DISTRIBUTED CLIENT-SERVER COLLABORATION APPLICATION

---

| | | |
|---|---|---|
| Advisor(s): | Nguyễn Lê Duy Lai | |
| Student(s): | Nguyễn Tiến Đạt | 2410712 |
| | Võ Văn Gia Khánh | 2411549 |
| | Nguyễn Hồng Phúc | 2412732 |

HO CHI MINH CITY, NOVEMBER 2026

# Contents

**1 Tổng quan hệ thống** **3**

**2 Kiến trúc hệ thống** **4**
2.1 Tổng quan . . . . . 4
2.2 Các thành phần chính . . . . . 4
2.3 Sơ đồ kiến trúc hệ thống . . . . . 5
2.4 Giao thức truyền thông . . . . . 5
2.5 Quản lý luồng thực thi . . . . . 5
2.6 Quản lý cộng đồng . . . . . 6

**3 Sequence Diagram** **7**
3.1 Luồng đăng nhập và kết nối P2P . . . . . 7
3.2 Luồng truyền file P2P . . . . . 7
3.3 Luồng broadcast file qua server . . . . . 8

**4 Khó khăn và giải pháp** **10**
4.1 Các vấn đề đã được giải quyết . . . . . 10
4.1.1 Phân mảnh gói tin TCP . . . . . 10
4.1.2 Đồng bộ hóa ghi lên socket dùng chung . . . . . 10
4.1.3 Phân luồng kết nối đến trên cùng một cổng . . . . . 11
4.1.4 Điều kiện tranh chấp trong thiết lập kết nối P2P . . . . . 11
4.1.5 Tính toàn vẹn dữ liệu file . . . . . 11
4.2 Vấn đề còn tồn tại . . . . . 12
4.2.1 Tiềm ẩn tranh chấp trong broadcast đa luồng . . . . . 12
4.2.2 Không có cơ chế bền vững (Persistence) . . . . . 12
4.2.3 Không có cơ chế heartbeat và phát hiện ngắt kết nối . . . . . 12

# 1 Tổng quan hệ thống

Hệ thống được xây dựng dựa lấy ý tưởng từ hai nền tảng giao tiếp và cộng tác phổ biến là Slack và Discord. Đây là những nền tảng cho phép nhiều người dùng kết nối, trao đổi thông tin và làm việc cùng nhau trong thời gian thực thông qua các không gian giao tiếp được tổ chức một cách có hệ thống.

Slack tập trung vào nhu cầu cộng tác trong môi trường làm việc, cung cấp các chức năng như không gian làm việc (Workspace), kênh giao tiếp (Channel), nhắn tin trực tiếp và chia sẻ tập tin. Trong khi đó, Discord hướng đến việc xây dựng cộng đồng trực tuyến thông qua các máy chủ (Server), kênh giao tiếp, quản lý thành viên và khả năng trao đổi thông tin theo thời gian thực.

Dựa trên những đặc điểm đó, hệ thống hướng đến việc kết hợp các mô hình giao tiếp tiêu biểu của cả hai nền tảng để xây dựng một ứng dụng cộng tác có khả năng hỗ trợ nhiều người dùng đồng thời. Người dùng có thể tạo và tham gia các cộng đồng, tổ chức hoạt động trao đổi thông tin theo từng kênh, gửi tin nhắn nhóm, nhắn tin trực tiếp và chia sẻ tập tin.

Về mặt kiến trúc mạng, hệ thống sử dụng mô hình kết hợp giữa Client–Server và Peer-to-Peer (P2P). Trong đó, mô hình Client–Server chịu trách nhiệm xử lý các chức năng tập trung như xác thực người dùng, quản lý cộng đồng, quản lý kênh và phân phối tin nhắn trong các cuộc hội thoại nhóm. Đối với giao tiếp trực tiếp giữa hai người dùng, hệ thống sử dụng mô hình P2P để thiết lập kết nối và trao đổi dữ liệu trực tiếp giữa các máy khách.

Mục tiêu của hệ thống không phải là xây dựng một bản sao hoàn chỉnh của Slack hay Discord, mà là vận dụng những ý tưởng cốt lõi của hai nền tảng này vào một ứng dụng cộng tác phục vụ mục đích học tập và thực hành các kiến thức về mạng máy tính. Thông qua quá trình triển khai, hệ thống tập trung thể hiện các kỹ thuật lập trình socket TCP, giao tiếp Client–Server, thiết lập kết nối P2P, xử lý đa luồng và truyền tập tin giữa các máy khách.



# 2 Kiến trúc hệ thống

## 2.1 Tổng quan

Hệ thống áp dụng mô hình kiến trúc *lai* (hybrid) kết hợp Client-Server tập trung và Peer-to-Peer (P2P) phân tán. Máy chủ trung tâm (Application Server) đảm nhiệm các chức năng xác thực, quản lý phiên, tra cứu địa chỉ ngang hàng, quản lý cộng đồng (Community), lưu trữ và phân phối file broadcast. Giao tiếp trực tiếp giữa hai client — bao gồm nhắn tin riêng tư và truyền file ngang hàng — được thực hiện qua kết nối TCP trực tiếp, không qua server, nhằm giảm tải và giảm độ trễ.

## 2.2 Các thành phần chính

- **Application Server**: Lắng nghe trên cổng TCP 8080 và cổng UDP 41234 (khám phá mạng). Mỗi kết nối TCP đến được xử lý bởi một `ClientHandler` chạy trên một *virtual thread* riêng biệt. Server duy trì các registry trong bộ nhớ: `UserSessionRegistry` (phiên đăng nhập), `ChatSessionRegistry` (phiên P2P), `CommunityRegistry` (cộng đồng) và `BroadcastFileRegistry` (file broadcast).

- **Client**: Mỗi tiến trình client duy trì một kết nối TCP điều khiển (control socket) đến server và một vòng lặp đọc gói tin (`receivePackets`) chạy trên virtual thread riêng. Client cũng khởi động `PeerListener` — một `ServerSocket` trên cổng ngẫu nhiên — để chấp nhận kết nối P2P đến từ các peer khác.

- **PeerListener**: Chạy trên virtual thread, lắng nghe kết nối TCP đến từ peer. Phân loại kết nối dựa trên gói tin đầu tiên nhận được: nếu là `PEER_HELLO` thì đây là phiên chat; nếu là `FILE_OFFER` thì đây là luồng truyền file.

- **PeerConnectionRegistry**: Quản lý tập hợp các kết nối P2P đang hoạt động (`ConcurrentHashMap<String, PeerConnection>`). Theo dõi peer đang chọn (active peer) và ngăn chặn kết nối trùng lặp qua tập `pendingLookups`.

- **FileStorageService**: Lưu trữ file broadcast phía server trên hệ thống tệp cục bộ. Hỗ trợ upload từ client gửi và download cho các client nhận thông qua token xác thực tạm thời.

- **AuthService**: Quản lý xác thực người dùng. Hiện tại dữ liệu tài khoản được lưu cứng trong bộ nhớ (hardcoded: alice/alice123, bob/bob123, admin1/password1); chưa có cơ sở dữ liệu bền vững.

## 2.3 Sơ đồ kiến trúc hệ thống

Hình 2.1 minh họa tổng quan cách các thành phần tương tác với nhau.

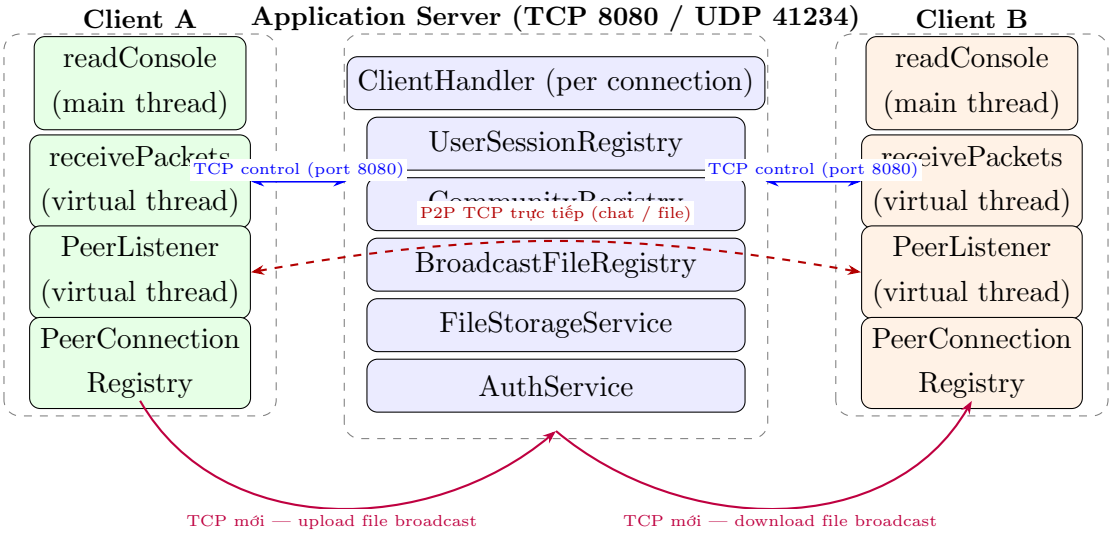



Figure 2.1: Kiến trúc tổng quan hệ thống Client-Server kết hợp P2P

## 2.4 Giao thức truyền thông

Toàn bộ giao tiếp giữa Client và Server sử dụng giao thức frame nhị phân tùy chỉnh với cấu trúc cố định:

$$\underbrace{[OP : 1\text{B}]}_{\text{OpCode}} \underbrace{[ReqID : 4\text{B}]}_{\text{Request ID}} \underbrace{[Len : 4\text{B}]}_{\text{Payload length}} \underbrace{[Data : N\text{B}]}_{\text{Payload}}$$

Phần header 9 byte được đọc trước, sau đó đọc chính xác $N$ byte payload bằng `DataInputStream.readFully()`. Cơ chế tương quan yêu cầu-phản hồi sử dụng `ConcurrentHashMap<Int` `CompletableFuture<Packet>` với timeout 10 giây, cho phép nhiều yêu cầu đồng thời trên một kết nối mà không bị nhầm lẫn phản hồi.

## 2.5 Quản lý luồng thực thi

Hệ thống sử dụng *virtual threads* (Project Loom, Java 21+) thay vì platform threads truyền thống để xử lý đồng thời:

- Mỗi kết nối TCP đến server tạo một virtual thread chạy `ClientHandler`.
- Mỗi `PeerConnection` có một virtual thread đọc gói tin độc lập.



- `PeerListener` chạy trên một virtual thread riêng; mỗi kết nối mới được phân phối ngay cho virtual thread tiếp theo.
- Trên client, `receivePackets` và `PeerListener` đều là virtual thread, song song với luồng chính đọc console.

Việc đồng bộ hóa ghi lên socket được thực hiện qua khóa đối tượng (object lock): `serverWriteLock` phía client, `synchronized(this)` trong `ClientHandler.sendPacket()`, và `synchronized(outputStream)` trong vòng lặp broadcast phía server.

## 2.6 Quản lý cộng đồng

Cộng đồng (Community) là khái niệm nhóm người dùng do server quản lý tập trung. `CommunityRegistry` lưu trữ các đối tượng `Community` trong `ConcurrentHashMap` theo ID. Mỗi Community có danh sách thành viên; chỉ thành viên mới nhận được tin nhắn broadcast và file broadcast của cộng đồng đó. Mọi trạng thái cộng đồng hiện tại chỉ tồn tại trong bộ nhớ và bị mất khi server khởi động lại.

# 3 Sequence Diagram

Phần này mô tả tuần tự các luồng hoạt động chính của hệ thống thông qua các sơ đồ trình tự (sequence diagram).

## 3.1 Luồng đăng nhập và kết nối P2P

Hình 3.1 mô tả toàn bộ quá trình từ khi Client A đăng nhập đến khi thiết lập kết nối P2P với Client B và trao đổi tin nhắn.

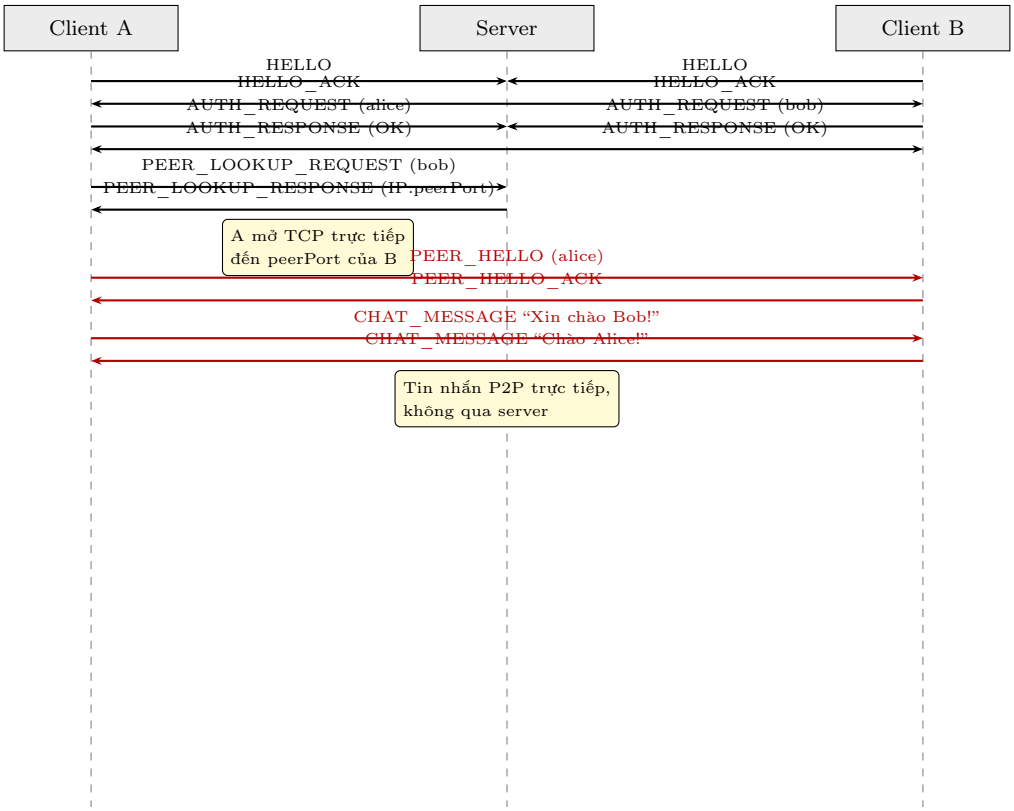



Figure 3.1: Sequence diagram: Đăng nhập và nhắn tin P2P trực tiếp

## 3.2 Luồng truyền file P2P

Hình 3.2 mô tả chi tiết quá trình truyền file trực tiếp giữa hai client sau khi đã thiết lập kết nối P2P. Toàn bộ luồng dữ liệu không đi qua server.

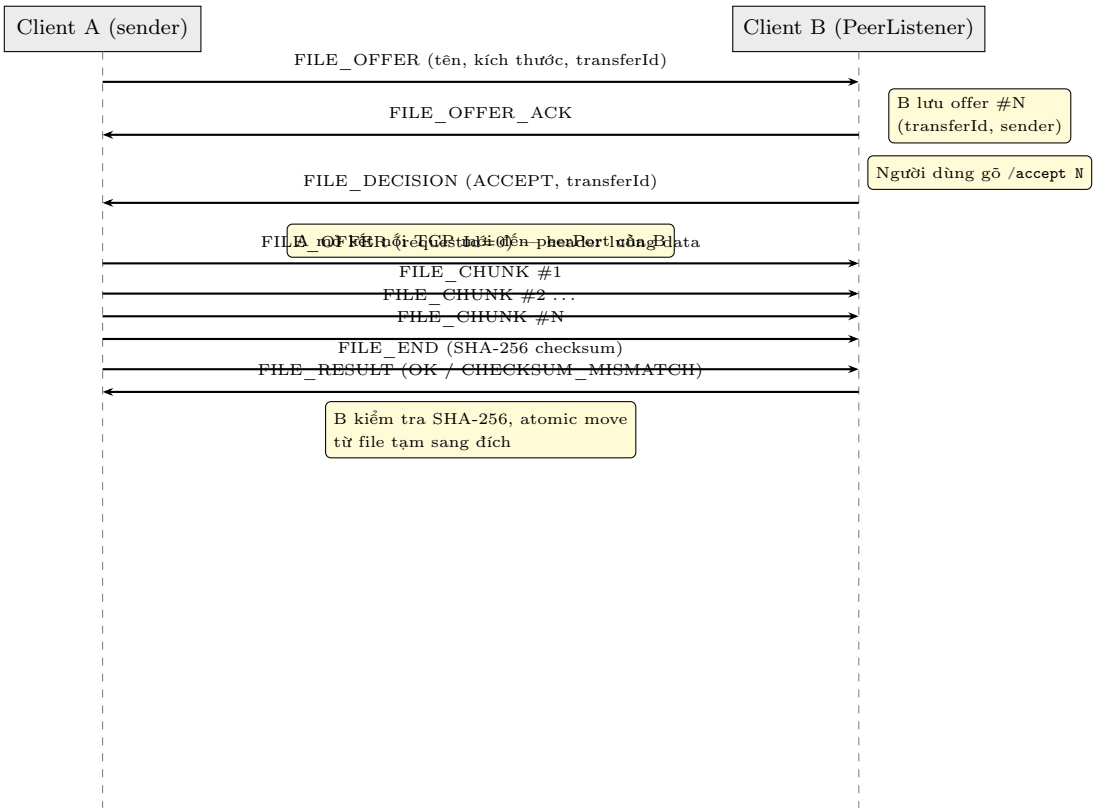



Figure 3.2: Sequence diagram: Truyền file P2P trực tiếp

## 3.3 Luồng broadcast file qua server

Hình 3.3 mô tả luồng broadcast file trong cộng đồng: Client A upload file lên server, server lưu trữ và phân phối đề nghị tải xuống đến các thành viên cùng cộng đồng. Mỗi client nhận tải file xuống qua một kết nối TCP riêng biệt đến cổng 8080.

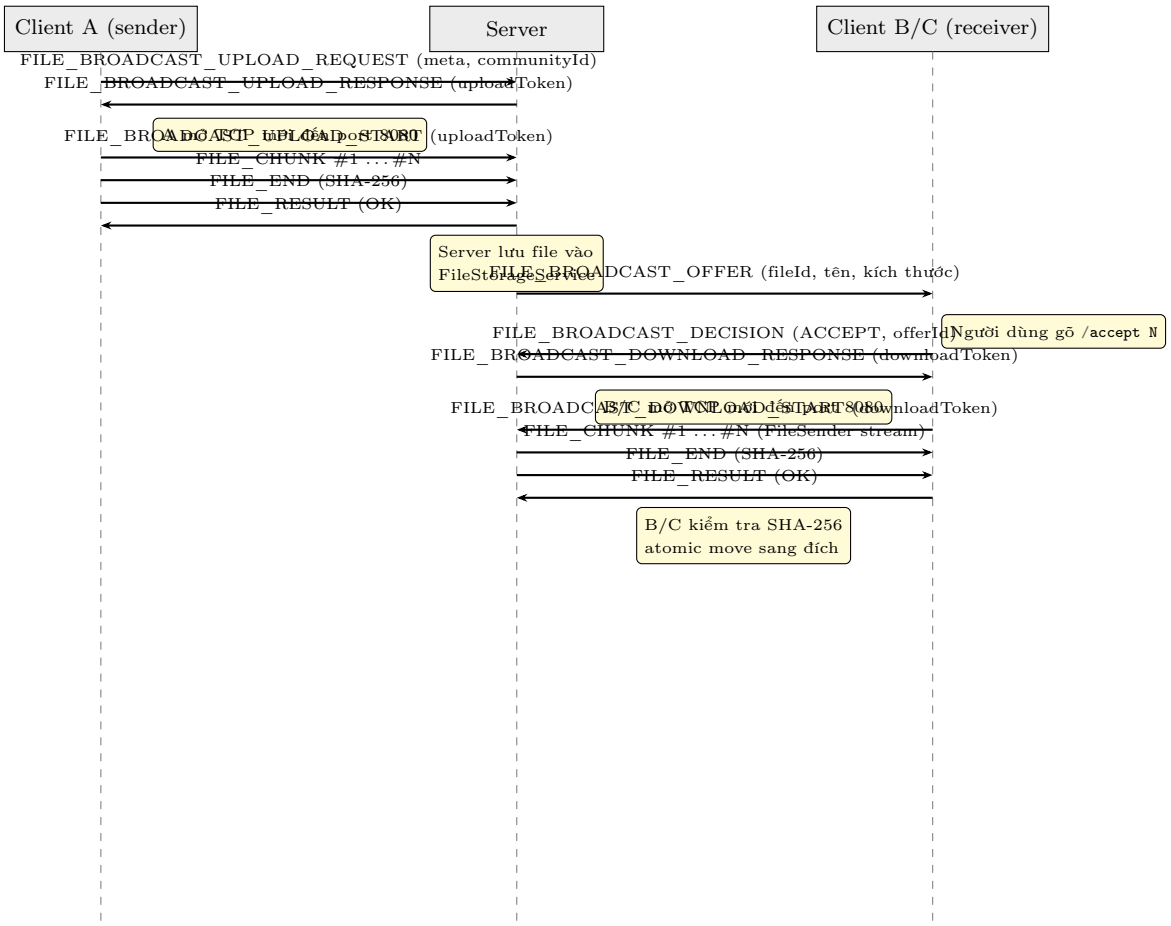



Figure 3.3: Sequence diagram: Broadcast file trong cộng đồng qua server trung tâm

# 4 Khó khăn và giải pháp

Trong quá trình thiết kế và triển khai hệ thống, nhóm gặp phải nhiều thách thức kỹ thuật liên quan đến lập trình socket, đồng bộ hóa luồng và truyền dữ liệu. Phần này trình bày các vấn đề thực tế được ghi nhận trong mã nguồn, phân biệt rõ giữa những vấn đề đã được giải quyết và những vấn đề còn tồn tại.

## 4.1 Các vấn đề đã được giải quyết

### 4.1.1 Phân mảnh gói tin TCP

TCP là giao thức hướng luồng (stream-oriented): không có ranh giới gói tin tự nhiên, dữ liệu có thể đến theo từng mảnh nhỏ hoặc bị ghép lại tùy ý. Nếu không có cơ chế phân tách gói tin rõ ràng, bên nhận sẽ không thể xác định điểm bắt đầu và kết thúc của mỗi thông điệp.

**Giải pháp:** Hệ thống sử dụng giao thức frame nhị phân tùy chỉnh với header 9 byte cố định: `[OpCode: 1B][RequestID: 4B][PayloadLen: 4B]`. Bên nhận luôn đọc đúng 9 byte header trước, lấy giá trị `PayloadLen`, rồi gọi `DataInputStream.readFully(buf, 0, len)` để đảm bảo đọc đủ $N$ byte payload, bất kể TCP có phân mảnh như thế nào. Cách tiếp cận này được áp dụng nhất quán trong cả `PacketCodec.decode()` và vòng lặp nhận file (`FileReceiver`).

### 4.1.2 Đồng bộ hóa ghi lên socket dùng chung

Nhiều virtual thread có thể cùng lúc muốn ghi lên cùng một `OutputStream` của một socket. Nếu không đồng bộ hóa, các byte từ hai gói tin khác nhau sẽ bị xen kẽ, gây lỗi phân tích giao thức ở phía nhận.

**Giải pháp:** Ba lớp khóa được áp dụng ở các vị trí khác nhau:

- Phía client: đối tượng khóa `serverWriteLock` bảo vệ toàn bộ thao tác ghi lên control socket.
- Phía server: `ClientHandler.sendPacket()` được đánh dấu `synchronized(this)`, đảm bảo chỉ một luồng ghi vào socket của client tại một thời điểm.
- Vòng lặp broadcast phía server: `synchronized(u.clientSocket().getOutputStream())` khóa trực tiếp trên output stream của từng client khi ghi gói tin broadcast.

### 4.1.3 Phân luồng kết nối đến trên cùng một cổng

Server lắng nghe trên một cổng TCP duy nhất (8080). Tuy nhiên, cổng này phải phục vụ ba loại kết nối khác nhau: phiên điều khiển thông thường, luồng upload file broadcast, và luồng download file broadcast. Tương tự, `PeerListener` phía client cũng phải phân biệt kết nối chat và kết nối dữ liệu file.

**Giải pháp:** Cả hai đều sử dụng chiến lược *đọc gói tin đầu tiên để phân loại*. Tại `ClientHandler.run()`: nếu gói đầu tiên là `HELLO` → khởi chạy phiên điều khiển; nếu là `FILE_BROADCAST_UPLOAD_START` → chuyển sang trình xử lý upload; nếu là `FILE_BROADCAST_DOWNLOAD_S` → chuyển sang trình xử lý download. Tại `PeerListener`: nếu là `PEER_HELLO` → khởi tạo kết nối chat; nếu là `FILE_OFFER` → chuyển cho `FileTransferController` xử lý luồng file.

### 4.1.4 Điều kiện tranh chấp trong thiết lập kết nối P2P

Khi hai client cùng thực hiện `/lookup` nhau gần như đồng thời, có thể xảy ra trường hợp cả hai đều gửi yêu cầu kết nối, dẫn đến hai kết nối P2P song song giữa cùng một cặp client.

**Giải pháp:** Trước khi gửi yêu cầu lookup, client kiểm tra `peerRegistry.isConnected(username)`. Nếu chưa kết nối, tên username được thêm vào `ConcurrentHashSet pendingLookups`; nếu đã có trong tập này (đang chờ phản hồi lookup), yêu cầu mới bị bỏ qua. Sau khi kết nối hoàn thành hoặc thất bại, username được xóa khỏi `pendingLookups`.

### 4.1.5 Tính toàn vẹn dữ liệu file

File truyền qua mạng có thể bị hỏng do lỗi đường truyền hoặc gián đoạn kết nối. Ngoài ra, file tạm thời (đang ghi) không được để lộ ra hệ thống tệp cho đến khi quá trình nhận hoàn tất.

**Giải pháp:** `FileSender` tính toán SHA-256 trực tiếp trong quá trình gửi chunk và đính kèm vào gói `FILE_END`. `FileReceiver` tính lại SHA-256 độc lập trong quá trình nhận và so sánh với giá trị nhận được. Nếu khớp, file tạm được di chuyển đến vị trí đích bằng `Files.move(..., StandardCopyOption.ATOMIC_MOVE)`; nếu không khớp, file tạm bị xóa và trả về `FILE_RESULT(CHECKSUM_MISMATCH)`.

## 4.2 Vấn đề còn tồn tại

### 4.2.1 Tiềm ẩn tranh chấp trong broadcast đa luồng

Phương thức `handleBroadcastMessage()` phía server truy cập `u.clientSocket().getOutputStrea` trực tiếp bằng `synchronized(u.clientSocket().getOutputStream())`. Tuy nhiên, nếu đồng thời có hai cuộc broadcast khác nhau (ví dụ từ hai cộng đồng khác nhau mà cùng một client là thành viên), mỗi broadcast sẽ khóa riêng lẻ trên output stream và vẫn có thể bị xen kẽ với các gói tin từ `ClientHandler.sendPacket()` vì `sendPacket()` khóa trên `this` (đối tượng `ClientHandler`), không phải trên output stream. Vấn đề này chưa được giải quyết triệt để và có thể dẫn đến lỗi phân tích giao thức trong điều kiện tải cao.

### 4.2.2 Không có cơ chế bền vững (Persistence)

Toàn bộ trạng thái hệ thống — bao gồm phiên đăng nhập, danh sách cộng đồng, thành viên, và metadata file broadcast — chỉ tồn tại trong bộ nhớ (`ConcurrentHashMap` trong RAM). Khi server khởi động lại, mọi dữ liệu đều bị mất. `AuthService` hiện tại sử dụng tài khoản hardcode, không có cơ sở dữ liệu hay file cấu hình. Đây là giới hạn thiết kế quan trọng cần giải quyết trước khi triển khai trong môi trường thực tế.

### 4.2.3 Không có cơ chế heartbeat và phát hiện ngắt kết nối

Hiện tại, server chỉ phát hiện client ngắt kết nối khi thao tác đọc/ghi trên socket ném ngoại lệ `IOException`. Không có cơ chế heartbeat (ping/pong) chủ động để phát hiện client bị treo hoặc mất kết nối âm thầm (silent disconnect). Điều này có thể dẫn đến trạng thái *zombie session* — client đã ngắt nhưng server vẫn coi là online.